

# Implementation and Verification of a CAN-FD Bus Transceiver in VHDL

Mads Richardt (s224948)

February 28, 2026

## Contents

|          |                                |          |
|----------|--------------------------------|----------|
| <b>1</b> | <b>Design and Architecture</b> | <b>1</b> |
| 1.1      | System Overview                | 1        |
| 1.2      | LLC Frame Format               | 3        |
| 1.3      | Interface Definition Tables    | 4        |
| 1.4      | Protocol-Driven Type System    | 10       |
| 1.5      | LLC Sub-layer                  | 10       |
| 1.5.1    | can_llc_tx                     | 10       |
| 1.5.2    | can_llc_rx                     | 10       |
| 1.6      | MAC Sub-layer                  | 10       |
| 1.6.1    | can_mac_tx                     | 11       |
| 1.6.1.1  | can_mac_tx Internal Interfaces | 11       |
| 1.6.1.2  | can_mac_fsm_tx                 | 14       |
| 1.6.1.3  | can_mac_ser_tx                 | 15       |
| 1.6.1.4  | can_mac_bs                     | 15       |
| 1.6.1.5  | can_mac_crc                    | 15       |
| 1.6.2    | can_mac_rx                     | 18       |
| 1.6.3    | can_mac Unified Wrapper        | 18       |
| 1.7      | FCE Sub-layer                  | 21       |
| 1.8      | PCS Sub-layer                  | 21       |
| 1.8.1    | can_pcs_tx                     | 21       |
| 1.8.2    | can_pcs_rx                     | 21       |

## 1 Design and Architecture

### 1.1 System Overview

A complete CAN node decomposes into a TX path and an RX path, coordinated by a shared Fault Confinement Entity (FCE) and Physical Medium Attachment (PMA) control, as shown in Figure 1. Each path spans three sub-layers - LLC (Section 1.5), MAC (Section 1.6), and PCS (Section 1.8) - with the LLC frame format defined in Section 1.2 and interface bundles defined in Section 1.3. A centralized types package (Section 1.4) defines all protocol constants and interface records. Within the MAC sub-layer, a unified can\_mac wrapper (Section 1.6.3) instantiates can\_mac\_tx, can\_mac\_rx, and can\_fce, merging their error signals internally so that the wrapper exposes only LLC and PCS interfaces for each path plus the FCE's LLC and PCS interfaces.

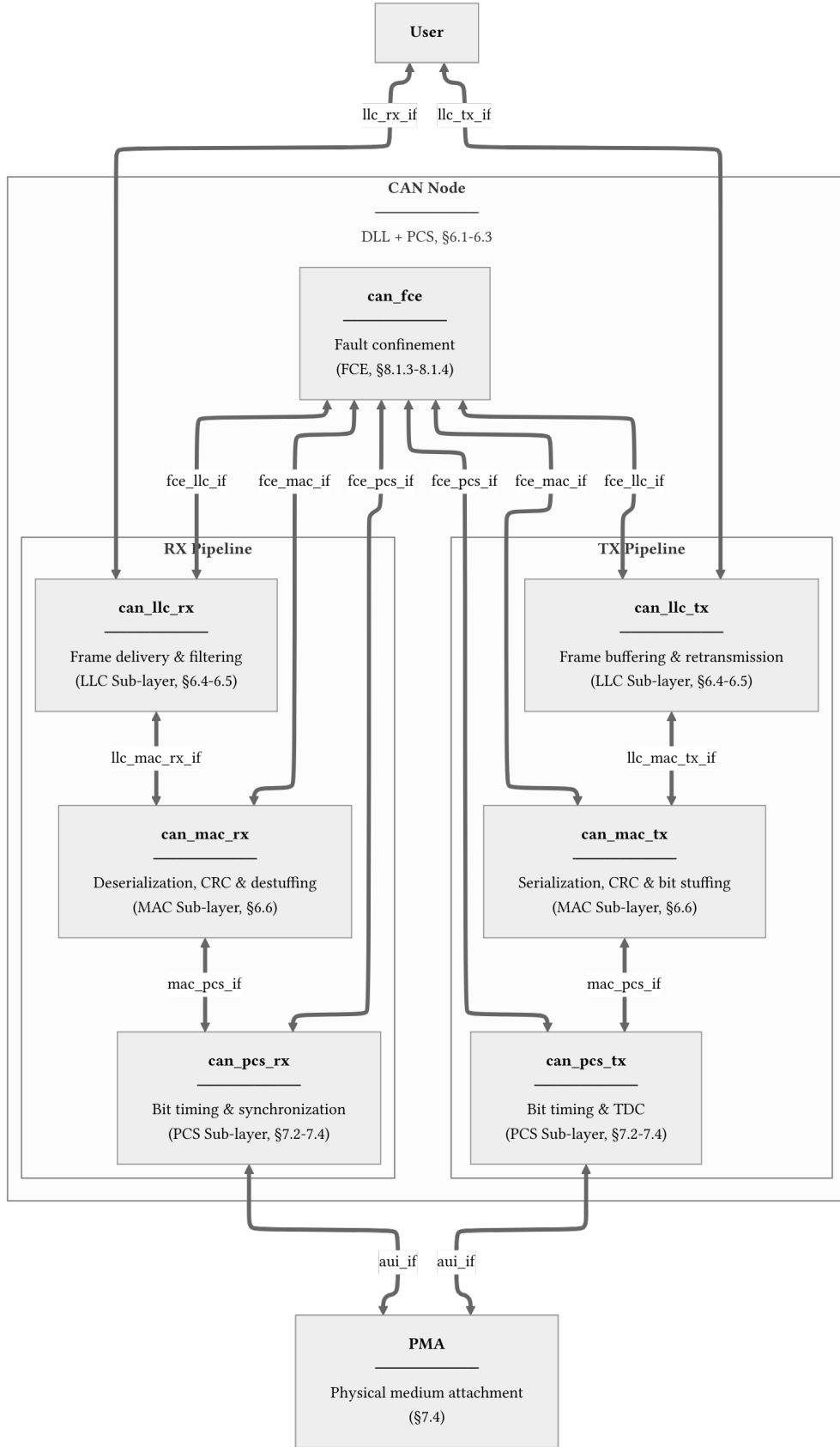


Figure 1: CAN node decomposition across LLC, MAC, PCS, FCE, and PMA boundaries. Interface definitions are provided for `llc_tx_if` (Table 2), `llc_rx_if` (Table 3), `llc_mac_tx_if` (Table 4), `llc_mac_rx_if` (Table 5), `mac_pcs_if` (Table 6), `auif` (Table 7), `fce_llc_if` (Table 8), `fce_mac_if` (Table 9), and `fce_pcs_if` (Table 10).

## 1.2 LLC Frame Format

The LLC Frame format used by the existing CAN-bus implementation (`can_bus_controller`) is depicted in Figure 2 with the ID byte format depicted in Table 1. A revised LLC Frame supporting FD is depicted in Figure 3. The revised format adds a 3-bit FMT [1, Sec. 6.4.3] field to the DLC byte (LLC Frame byte 4), expands the data field to 64 bytes, and repurposes reserved bits in the last byte for BRS and ESI flags.

The FMT encodes the supported frame formats as ‘000’ = CB, ‘100’ = CE, ‘010’ = FB, ‘110’ = FE. Accordingly, for the frame content to be self-consistent the IDE bit must be set to 0 for FMT = CB/FB and 1 for FMT = CE/FE. The revised format is designed to be backward compatible. An implementation that only supports Classic frames can simply ignore the FMT bits and treat all frames as Classic, while an implementation that supports FD can use the FMT field and the additional control bits (BRS and ESI) to distinguish frame types without affecting the existing ID and data field structure.

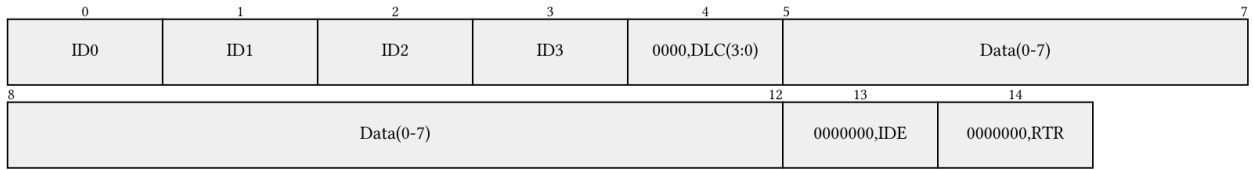


Figure 2: Current LLC frame format (15 bytes). ID byte encoding is defined in Table 1.

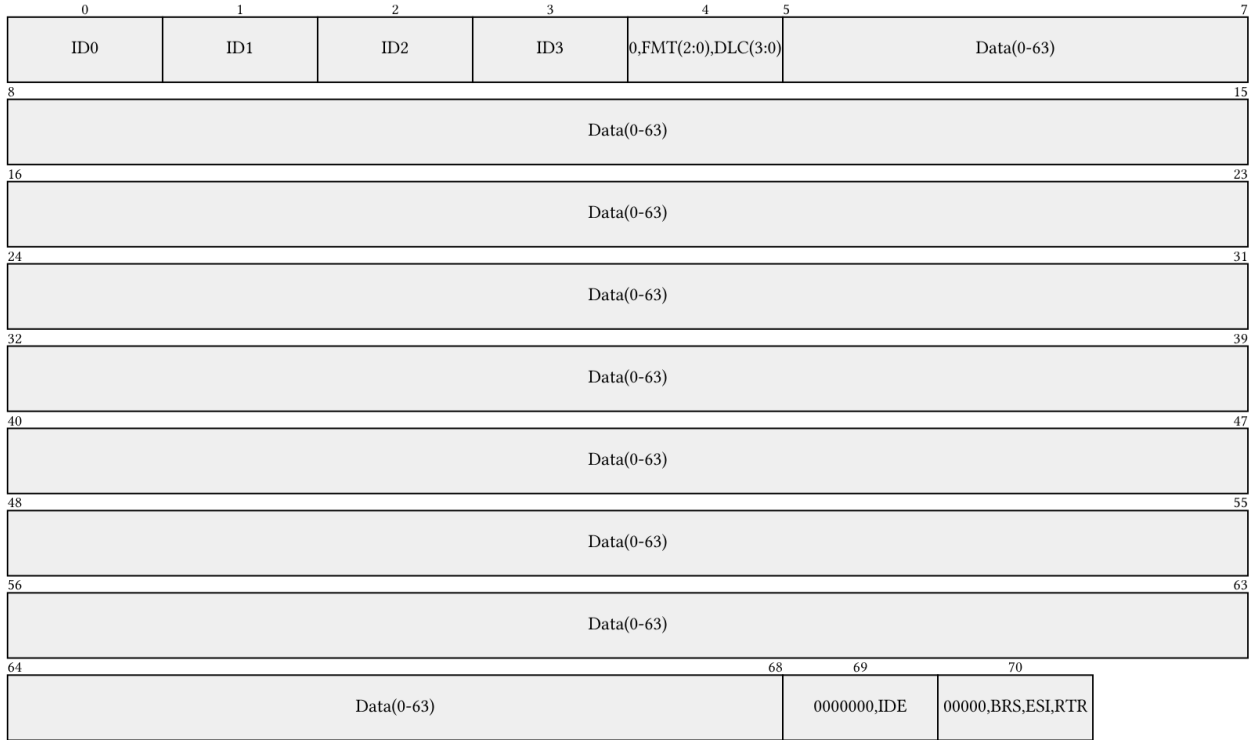


Figure 3: Revised LLC frame format with FD support, shown at maximum length (71 bytes). The FMT field selects frame type; BRS and ESI repurpose reserved bits in the final byte. ID byte encoding is defined in Table 1.

Table 1: ID byte encoding for the LLC frame formats shown in Figure 2 and Figure 3.

| Format   | Byte ID0               | Byte ID1    | Byte ID2                | Byte ID3  |
|----------|------------------------|-------------|-------------------------|-----------|
| Basic    | '00000000'             | '00000000'  | '00000' &<br>'ID(10-8)' | 'ID(7-0)' |
| Extended | '000' &<br>'ID(28:24)' | 'ID(23-16)' | 'ID(15-8)'              | 'ID(7-0)' |

### 1.3 Interface Definition Tables

The interface bundles shown in Figure 1 are defined in full below, with each signal mapped to its corresponding ISO 11898-1 service primitive, [1]. This normative anchoring provides a direct traceability path from protocol clauses to VHDL ports. The types used in these interfaces are defined in Section 1.4.

All module interfaces use `std_logic` and `std_logic_vector` exclusively, as mandated for synthesis compatibility. Protocol semantics such as polarity, frame format, and transfer status are encoded as named `std_logic/std_logic_vector` constants defined in `pk_can_types` (Section 1.4). The LLC-User interface (`llc_tx_if`, `llc_rx_if`) is implemented as an Avalon-ST byte stream to maintain backward compatibility with the existing CAN bus controller, which uses the same `valid/ready/startofpacket` handshake convention.

Table 2: Interface definition for `llc_tx_if`. Implements `L_Data.Request` as an Avalon-ST byte stream. `L_Data.AbortRequest` is included for complete ISO service coverage but is not yet implemented.

| ISO ref.  | ISO symbol                       | ISO payload       | ISO semantics                     | Direction   | Implementation mapping                                                                                                         | Implementation notes                                     |
|-----------|----------------------------------|-------------------|-----------------------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| 6.4.5.5.2 | <code>L_Data.Request</code>      | LLC Frame, Handle | Submit LLC frame for transmission | User -> LLC | <code>llc_tx_if.data (byte_t)</code> ,<br><code>llc_tx_if.valid (std_logic)</code> ,<br><code>llc_tx_if.sop (std_logic)</code> | valid high with byte on data; sop asserted on first byte |
| 6.4.5.5.2 | <code>L_Data.Request</code>      | LLC Frame, Handle | Submit LLC frame for transmission | User -> LLC | <code>llc_tx_if.data (byte_t)</code> ,<br><code>llc_tx_if.valid (std_logic)</code>                                             | valid high; sop and eop deasserted on intermediate bytes |
| 6.4.5.5.2 | <code>L_Data.Request</code>      | LLC Frame, Handle | Submit LLC frame for transmission | User -> LLC | <code>llc_tx_if.data (byte_t)</code> ,<br><code>llc_tx_if.valid (std_logic)</code> ,<br><code>llc_tx_if.eop (std_logic)</code> | valid high with byte on data; eop asserted on last byte  |
| 6.4.5.5.2 | <code>L_Data.Request</code>      |                   | Flow control                      | LLC -> User | <code>llc_tx_if.ready (std_logic)</code>                                                                                       | Asserted by LLC when able to consume next byte           |
| 6.4.5.5.3 | <code>L_Data.AbortRequest</code> | Handle            | Abort pending frame transfer      | User -> LLC | <code>llc_tx_if.abort_request (std_logic)</code>                                                                               | Pulse when user wants to cancel an in-progress transfer  |

Table 3: Interface definition for `llc_rx_if`. Implements `L_Data.Indication` as an Avalon-ST byte stream. Timestamp is included for complete ISO service coverage but is not yet implemented.

| ISO ref.  | ISO symbol                     | ISO payload          | ISO semantics                      | Direction   | Implementation mapping                                                                                                         | Implementation notes                                     |
|-----------|--------------------------------|----------------------|------------------------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| 6.4.5.5.5 | <code>L_Data.Indication</code> | LLC Frame, Timestamp | Deliver received LLC frame to user | LLC -> User | <code>llc_rx_if.data (byte_t)</code> ,<br><code>llc_rx_if.valid (std_logic)</code> ,<br><code>llc_rx_if.sop (std_logic)</code> | valid high with byte on data; sop asserted on first byte |
| 6.4.5.5.5 | <code>L_Data.Indication</code> | LLC Frame, Timestamp | Deliver received LLC frame to user | LLC -> User | <code>llc_rx_if.data (byte_t)</code> ,<br><code>llc_rx_if.valid (std_logic)</code>                                             | valid high; sop and eop deasserted on intermediate bytes |
| 6.4.5.5.5 | <code>L_Data.Indication</code> | LLC Frame, Timestamp | Deliver received LLC frame to user | LLC -> User | <code>llc_rx_if.data (byte_t)</code> ,<br><code>llc_rx_if.valid (std_logic)</code> ,<br><code>llc_rx_if.eop (std_logic)</code> | valid high with byte on data; eop asserted on last byte  |

| ISO ref.  | ISO symbol        | ISO payload                        | ISO semantics                          | Direction   | Implementation mapping                            | Implementation notes                            |
|-----------|-------------------|------------------------------------|----------------------------------------|-------------|---------------------------------------------------|-------------------------------------------------|
| 6.4.5.5.5 | L_Data.Indication |                                    | Flow control                           | User -> LLC | llc_rx_if.ready (std_logic)                       | Asserted by user when able to consume next byte |
| 6.4.5.5.4 | L_Data.Confirm    | Transfer_Status, Timestamp, Handle | Report outcome of prior L_Data.Request | LLC -> User | llc_rx_if.transfer_status (std_logic_vector(2:0)) | Issued on frame completion, loss, or error      |

Table 4: Interface definition for llc\_mac\_tx\_if. Frame length is self-describing from the DLC config bytes; the MAC serializer does not consume eop.

| ISO ref.     | ISO symbol                    | ISO payload     | ISO semantics                               | Direction  | Implementation mapping                                                                      | Implementation notes                                            |
|--------------|-------------------------------|-----------------|---------------------------------------------|------------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| 6.3, 6.6.4.2 | DLL SDU                       | LLC Frame       | Transfer LLC frame to MAC for serialization | LLC -> MAC | llc_mac_tx_if.data (byte_t), llc_mac_tx_if.valid (std_logic), llc_mac_tx_if.sop (std_logic) | valid high with byte on data; sop marks first byte of new frame |
| 6.3, 6.6.4.2 | DLL SDU                       |                 | Flow control                                | MAC -> LLC | llc_mac_tx_if.ready (std_logic)                                                             | Asserted by MAC serializer when able to consume next byte       |
| 6.6.4.2      | Interface control information | Transfer_Status | Report frame transmission outcome           | MAC -> LLC | llc_mac_tx_if.transfer_status (std_logic_vector(2:0))                                       | Updated by MAC on completion, loss, or error                    |

Table 5: Interface definition for llc\_mac\_rx\_if. Carries the reconstructed LLC frame from can\_mac\_rx to can\_llc\_rx (Section 1.6.2) as an Avalon-ST byte stream. The RX FSM stores received bits in an internal frame array during reception and streams the completed frame after EOF.

| ISO ref.       | ISO symbol | ISO payload             | ISO semantics                           | Direction  | Implementation mapping                                                                                                     | Implementation notes                                                                        |
|----------------|------------|-------------------------|-----------------------------------------|------------|----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| 6.6.4.3, 6.6.9 | DLL SDU    | Reconstructed LLC Frame | Transfer reconstructed LLC frame to LLC | MAC -> LLC | llc_mac_rx_if.data (byte_t), llc_mac_rx_if.valid (std_logic), llc_mac_rx_if.sop (std_logic), llc_mac_rx_if.eop (std_logic) | Avalon-ST byte stream; the RX FSM streams the stored llc_frame array during the quiet phase |
| 6.6.4.3, 6.6.9 | DLL SDU    |                         | Flow control                            | LLC -> MAC | llc_mac_rx_if.ready (std_logic)                                                                                            | Asserted by LLC when able to consume next byte                                              |

Table 6: Interface definition for `mac_pcs_if`. The `D_Transmit` status is signalled via a dedicated `use_data_rate` flag rather than encoding it in a semantic bit name. The PCS switches between nominal and data-phase bit timing based on this flag, keeping the interface to plain `std_logic` signals.

| ISO ref.     | ISO symbol                          | ISO payload              | ISO semantics                          | Direction  | Implementation mapping                                                                                                                                                                               | Implementation notes                                                                                                      |
|--------------|-------------------------------------|--------------------------|----------------------------------------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| 7.2.1, 7.2.2 | <code>PCS_Data.Request</code>       | <code>Output_Unit</code> | Present next bit for transmission      | MAC -> PCS | <code>mac_pcs_if.polarity (std_logic)</code> ,<br><code>mac_pcs_if.valid (std_logic)</code>                                                                                                          | MAC holds polarity stable; PCS samples at each bit boundary autonomously                                                  |
| 7.2.1, 7.2.5 | <code>PCS_Status.Transmitter</code> | <code>D_Transmit</code>  | Indicate FD data-phase interval to PCS | MAC -> PCS | <code>mac_pcs_if.use_data_rate (std_logic)</code> ,<br><code>mac_pcs_if.start_tdc (std_logic)</code>                                                                                                 | <code>use_data_rate</code> asserted during FD data phase; <code>start_tdc</code> pulsed at BRS to begin delay measurement |
| 7.2.1, 7.2.3 | <code>PCS_Data.Indicate</code>      | <code>Input_Unit</code>  | Indicate bus polarity at sample point  | PCS -> MAC | <code>mac_pcs_if.bus_polarity (std_logic)</code> ,<br><code>mac_pcs_if.sp (std_logic)</code> ,<br><code>mac_pcs_if.ssp (std_logic)</code> ,<br><code>mac_pcs_if.fifo_index (t_fifo_index_vec)</code> | <code>sp</code> pulses once per nominal sample point; <code>ssp</code> pulses once per secondary sample point for TDC     |
| 7.2.1, 7.2.6 | <code>PCS_Status.Receiver</code>    | <code>D_Receive</code>   | Indicate FD data-phase interval to PCS | MAC -> PCS | not yet implemented                                                                                                                                                                                  | Asserted during FD data-phase reception                                                                                   |

Table 7: Interface definition for `auif_if`. All signals are `std_logic`, as this interface crosses from the ISO protocol domain into the physical medium.

| ISO ref.         | ISO symbol                          | ISO payload                            | ISO semantics                | Direction  | Implementation mapping                           | Implementation notes                             |
|------------------|-------------------------------------|----------------------------------------|------------------------------|------------|--------------------------------------------------|--------------------------------------------------|
| 7.4.2.1          | <code>output symbol</code>          | <code>Dominant/recessive symbol</code> | Drive physical output symbol | PCS -> PMA | <code>auif_if.tx (std_logic)</code>              | Updated on each <code>Output_Unit</code> request |
| 7.4.2.2, 8.1.3.4 | <code>bus_off symbol</code>         | <code>Bus-off control</code>           | Switch node off bus          | PCS -> PMA | <code>auif_if.bus_off (std_logic)</code>         | On <code>Bus_off_request</code> from FCE         |
| 7.4.2.3, 8.1.3.4 | <code>bus_off_release symbol</code> | <code>Bus-off release control</code>   | Release node from bus-off    | PCS -> PMA | <code>auif_if.bus_off_release (std_logic)</code> | On <code>Bus_off_release_request</code> from FCE |

| ISO ref. | ISO symbol   | ISO payload               | ISO semantics                  | Direction  | Implementation mapping | Implementation notes       |
|----------|--------------|---------------------------|--------------------------------|------------|------------------------|----------------------------|
| 7.4.3    | input symbol | Dominant/recessive symbol | Indicate physical input symbol | PMA -> PCS | auif.rx (std_logic)    | Continuously driven by PMA |

Table 8: Interface definition for fce\_llc\_if. Carries bus-off status and mode-request handshake between the FCE and LLC layers [1, Sec. 8.1.3.2].

| ISO ref. | ISO symbol           | ISO payload    | ISO semantics                   | Direction  | Implementation mapping                      | Implementation notes           |
|----------|----------------------|----------------|---------------------------------|------------|---------------------------------------------|--------------------------------|
| 8.1.3.2  | Normal_mode_request  | Mode request   | Request reset to normal mode    | LLC -> FCE | fce_llc_if.normal_mode_request (std_logic)  | Issued on startup/restart      |
| 8.1.3.2  | Normal_mode_response | Mode response  | Acknowledge normal-mode request | FCE -> LLC | fce_llc_if.normal_mode_response (std_logic) | Returned after FCE processing  |
| 8.1.3.2  | Bus_off              | Bus-off status | Indicate node is bus-off        | FCE -> LLC | fce_llc_if.bus_off (std_logic)              | Asserted on bus-off transition |

∞

Table 9: Interface definition for fce\_mac\_if. The MAC reports error events and frame outcomes to the FCE; the FCE returns the current error-active/passive state to the MAC [1, Sec. 8.1.3.3].

| ISO ref. | ISO symbol          | ISO payload              | ISO semantics                   | Direction  | Implementation mapping                             | Implementation notes                  |
|----------|---------------------|--------------------------|---------------------------------|------------|----------------------------------------------------|---------------------------------------|
| 8.1.3.3  | Transmit/receive    | Transfer mode context    | Report current TX/RX context    | MAC -> FCE | fce_mac_if.transmitting (std_logic)                | Updated with MAC transfer context     |
| 8.1.3.3  | Error               | Error event              | Report detected protocol error  | MAC -> FCE | fce_mac_if.error (std_logic)                       | Pulse on bit/stuff/CRC/form/ACK error |
| 8.1.3.3  | Primary_error       | Primary error event      | Report primary error condition  | MAC -> FCE | fce_mac_if.primary_error (std_logic)               | Pulse on primary error condition      |
| 8.1.3.3  | Error/overload flag | EF/OF state              | Report EF/OF transmission state | MAC -> FCE | fce_mac_if.sending_error_overload_flag (std_logic) | Asserted during EF/OF transmission    |
| 8.1.3.3  | Counters_unchanged  | Counter-update qualifier | Qualify counter exception path  | MAC -> FCE | fce_mac_if.counters_unchanged (std_logic)          | Asserted on rule-c exception cases    |

| ISO ref. | ISO symbol               | ISO payload               | ISO semantics                            | Direction  | Implementation mapping                          | Implementation notes                 |
|----------|--------------------------|---------------------------|------------------------------------------|------------|-------------------------------------------------|--------------------------------------|
| 8.1.3.3  | Error_delimiter_too_late | Late delimiter event      | Report late error-delimiter condition    | MAC -> FCE | fce_mac_if.error_delimiter_too_late (std_logic) | Asserted on late delimiter condition |
| 8.1.3.3  | Successful_transfer      | Transfer completion event | Report successful TX/RX completion       | MAC -> FCE | fce_mac_if.successful_transfer (std_logic)      | Pulse on successful frame transfer   |
| 8.1.3.3  | Error_passive_response   | State response            | Report entry into error-passive state    | MAC -> FCE | fce_mac_if.error_passive_response (std_logic)   | On state transition completion       |
| 8.1.3.3  | Error_active_response    | State response            | Report return to error-active state      | MAC -> FCE | fce_mac_if.error_active_response (std_logic)    | On state transition completion       |
| 8.1.3.3  | Error_passive_request    | State request             | Request MAC enter error-passive state    | FCE -> MAC | fce_mac_if.error_passive_request (std_logic)    | On TEC/REC threshold crossing        |
| 8.1.3.3  | Error_active_request     | State request             | Request MAC return to error-active state | FCE -> MAC | fce_mac_if.error_active_request (std_logic)     | On TEC/REC recovery                  |

Table 10: Interface definition for fce\_pcs\_if. Carries bus-off and bus-off-release request/response handshakes between the FCE and PCS layers [1, Sec. 8.1.3.4].

| ISO ref. | ISO symbol               | ISO payload              | ISO semantics                       | Direction  | Implementation mapping                          | Implementation notes            |
|----------|--------------------------|--------------------------|-------------------------------------|------------|-------------------------------------------------|---------------------------------|
| 8.1.3.4  | Bus_off_request          | Bus-off request          | Request node switch-off from bus    | FCE -> PCS | fce_pcs_if.bus_off_request (std_logic)          | On bus-off transition condition |
| 8.1.3.4  | Bus_off_release_request  | Bus-off release request  | Request node re-enable from bus-off | FCE -> PCS | fce_pcs_if.bus_off_release_request (std_logic)  | On restart/reintegration        |
| 8.1.3.4  | Bus_off_response         | Bus-off response         | Acknowledge bus-off request         | PCS -> FCE | fce_pcs_if.bus_off_response (std_logic)         | Returned after bus-off action   |
| 8.1.3.4  | Bus_off_release_response | Bus-off release response | Acknowledge bus-off-release request | PCS -> FCE | fce_pcs_if.bus_off_release_response (std_logic) | Returned after release action   |

## 1.4 Protocol-Driven Type System

Two VHDL packages centralize the shared definitions used across the design. All module interfaces use `std_logic` and `std_logic_vector` exclusively - protocol concepts such as polarity, frame format, and transfer status are encoded as named constants. Enumeration types such as FSM state types are declared locally within each module's architecture, keeping the packages free of module-specific types.

`pk_can_types` (`can_types_p.vhd`) is the primary design package, organized into the following sections:

1. **Protocol constants** - bus polarity (`c_dominant = '0'`, `c_recessive = '1'`), frame field widths (base ID 11 bits, extended ID 18 bits, DLC 4 bits, EOF 7 bits), bit stuffing parameters (`c_stuff_width = 5`), CRC polynomial vectors and selector constants (CRC-15, CRC-17, CRC-21), transfer status encodings (`c_ongoing`, `c_transmitted`, `c_aborted`, `c_lost_arb`, `c_disturbed`), error signalling widths, inter-frame spacing widths, and FCE counter thresholds.
2. **Bit timing subtypes** - constrained natural ranges matching ISO Table 12 [1, Sec. 7.3.2]: prescaler (1-32), propagation segments, phase segments, and SSP offset (1-63).
3. **Composite types** - the sole composite type is `t_llc_metadata`, a record capturing the six LLC config byte fields (IDE, FDF, DLC, FTYP, BRS, ESI) extracted by the serializer and held stable for the duration of a frame.
4. **Interface records** - typed bundles for each inter-module boundary (serializer-FSM, LLC-MAC TX/RX, MAC-PCS, FSM-bit stuffer, FSM-CRC, MAC-FCE, LLC-FCE, FCE-PCS), each paired with a reset constant. These are defined in full in Section 1.3.
5. **LLC frame format** - array types and config byte bit-position constants for both the internal LLC frame format (variable length, streamed to the serializer) and the legacy 71-byte user-facing format.
6. **Utility functions** - `dlc_to_data_length` (ISO Table 5 DLC-to-byte-count conversion), `f_to_gray` (binary-to-Gray encoding for the SBC field), and `f_calc_parity` (XOR parity for SBC).

`can_tb_p` (`can_tb_p.vhd`) is the testbench utility package, extracted from `pk_can_types` to separate simulation-only code from synthesizable definitions. It provides CRC reference calculation, metadata extraction, and bus stream reference model functions used by the testbenches for expected-value computation.

## 1.5 LLC Sub-layer

Responsible for frame buffering and retransmission management. It provides an Avalon-ST interface to the user application and communicates with the FCE to handle retransmission limits and error status reporting.

### 1.5.1 can\_llc\_tx

`can_llc_tx` buffers an incoming LLC frame from the user, streams it byte-by-byte to the MAC serializer, and manages retransmission on bus disturbance up to the ISO-mandated limit [1, Sec. 6.4.5] and 6.5.

### 1.5.2 can\_llc\_rx

`can_llc_rx` receives a reconstructed LLC frame from the MAC layer, applies acceptance filtering, and delivers accepted frames to the user over the `llc_rx_if` Avalon-ST stream [1, Sec. 6.4.5].

## 1.6 MAC Sub-layer

The MAC sub-layer is the core of the protocol logic, responsible for bit serialization, CRC generation, bit stuffing, and frame-level error detection. It coordinates closely with the FCE (Section 1.7) for error counter

management and node-state transitions (Error Active/Passive/Bus Off), and with the PCS (Section 1.8) for sample-point-driven bit output.

The earlier CAN bus controller concentrated TX, RX, MAC, and FCE logic in a single monolithic FSM (`can_fsm`), with the sub-functions - serialization (`can_ast_to_serial`), bit stuffing (`can_stuff_bit_gen`), and CRC (`gen_crc`) - implemented in satellite modules driven directly by it. PCS timing logic was implemented in `can_node_clock`, which fed sample-point and transmit pulses into `can_fsm` - a strategy retained in the current design.

The current design restructures these modules around the ISO 11898-1 [1] layer boundaries. On the TX side the mapping is direct: `can_ast_to_serial` becomes `can_mac_ser_tx` (Section 1.6.1.3), `can_stuff_bit_gen` becomes `can_mac_bs` (Section 1.6.1.4), `gen_crc` becomes `can_mac_crc` (Section 1.6.1.5), and `can_node_clock` becomes `can_pcs_tx` (Section 1.8.1). The bit stuffer and CRC engine are shared between the TX and RX paths - each path instantiates its own copy of the same `can_mac_bs` and `can_mac_crc` entities, reusing the bit stuffer for destuffing and the CRC engine for CRC checking on the RX side. The RX FSM (`can_mac_fsm_rx`) stores received bits directly in an internal frame array and streams the completed frame to the LLC during the quiet phase, eliminating the need for a separate deserializer module. Fault confinement, previously embedded in `can_fsm`, is separated into its own entity (`can_fce`) and wired through the unified `can_mac` wrapper (Section 1.6.3).

### 1.6.1 `can_mac_tx`

The `can_mac_tx` (Figure 4) is composed of four main components:

1. **`can_mac_ser_tx`**: Converts LLC bytes into a serial bit stream, extracts LLC metadata from config bytes
2. **`can_mac_fsm_tx`**: Coordinates frame transmission with per-field state granularity, controls all sub-modules
3. **`can_mac_bs`**: Shared bit stuffer implementing both dynamic and fixed bit stuffing modes
4. **`can_mac_crc`**: CRC engine with three parallel generators (CRC-15, CRC-17, CRC-21) and dual data feeds for CC/FD

**1.6.1.1 `can_mac_tx` Internal Interfaces** The interfaces between MAC sub-components are implementation-defined and carry no direct ISO service primitive mapping. Table 11, Table 12, and Table 13 define each bidirectional bundle; the Direction column identifies the driving component for each field. The bit stuffer and CRC engine interfaces are shared between the TX and RX paths - each path instantiates its own copy wired through identical record types.

Table 11: Interface definition for `can_mac_ser_fsm_if`, connecting `can_mac_ser_tx` and `can_mac_fsm_tx` (see Figure 4).

| field | type      | direction  | description                                                 |
|-------|-----------|------------|-------------------------------------------------------------|
| data  | std_logic | ser -> fsm | current bit polarity; FSM reads this when valid is asserted |
| valid | std_logic | ser -> fsm | asserted while a bit is available for the FSM to consume    |

| field           | type                  | direction  | description                                                                                                             |
|-----------------|-----------------------|------------|-------------------------------------------------------------------------------------------------------------------------|
| llc_metadata    | t_llc_metadata        | ser -> fsm | LLC config byte fields (IDE, FDF, DLC, FTYP, BRS, ESI) extracted by the serializer; valid for the lifetime of the frame |
| ready           | std_logic             | fsm -> ser | pulsed when the FSM has consumed the current bit; advances the serializer to the next bit                               |
| transfer_status | std_logic_vector(2:0) | fsm -> ser | frame outcome; any non-c_ongoing value terminates serialization                                                         |

Table 12: Interface definition for `can_mac_fsm_bs_if`, connecting `can_mac_fsm_tx/can_mac_fsm_rx` and `can_mac_bs` (see Figure 4). The bit stuffer is reset by a dedicated `bs_rst` signal from the FSM rather than a field in this record.

| field                 | type                  | direction | description                                                                    |
|-----------------------|-----------------------|-----------|--------------------------------------------------------------------------------|
| data                  | std_logic             | fsm -> bs | bit polarity fed into the bit stuffer                                          |
| valid                 | std_logic             | fsm -> bs | pulsed when a new bit is presented to the stuffer                              |
| fixed_bit_stuffing_en | std_logic             | fsm -> bs | when high, the bit stuffer operates in fixed bit stuffing mode (FD CRC region) |
| data                  | std_logic             | bs -> fsm | polarity of the required stuff bit                                             |
| valid                 | std_logic             | bs -> fsm | asserted when a stuff bit insertion is required                                |
| stuff_bit_count       | std_logic_vector(3:0) | bs -> fsm | gray-coded stuff bit count with parity for the FD SBC field                    |

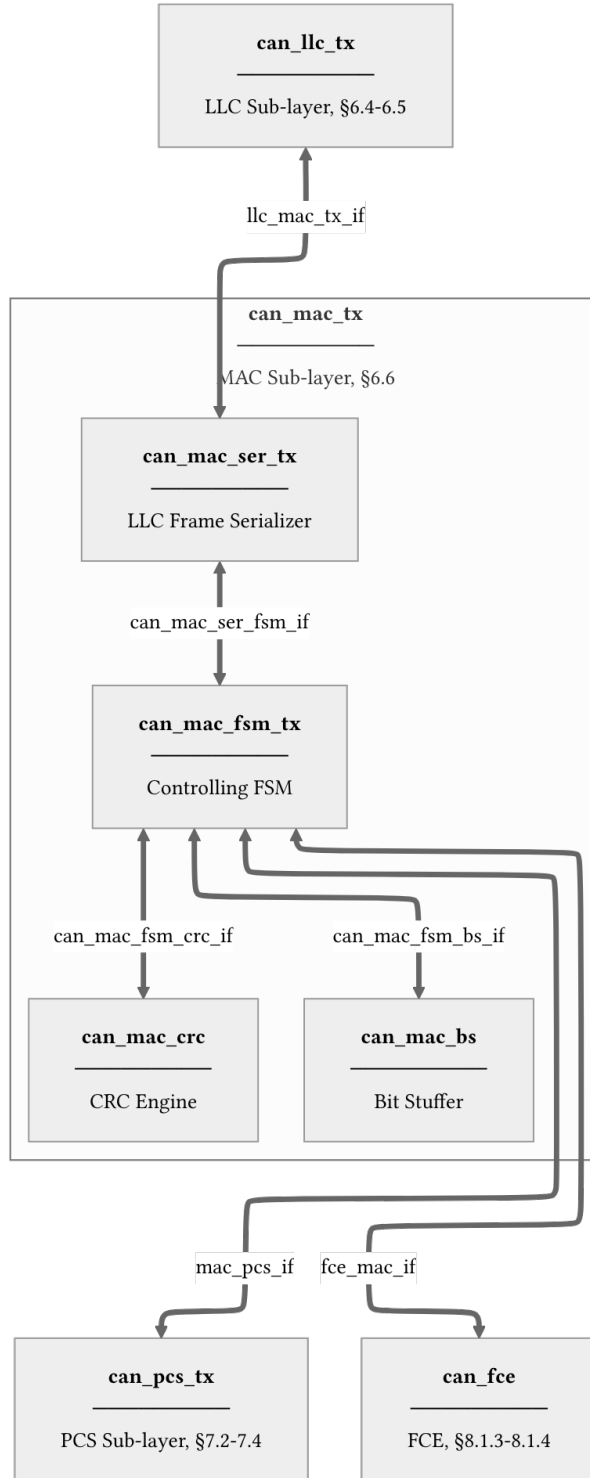


Figure 4: **can\_mac\_tx** architecture showing internal component interconnections and external interfaces. Internal interface definitions are provided for **can\_mac\_ser\_fsm\_if** (Table 11), **can\_mac\_fsm\_bs\_if** (Table 12), and **can\_mac\_fsm\_crc\_if** (Table 13). The bit stuffer and CRC engine are the same entities reused in **can\_mac\_rx**.

Table 13: Interface definition for `can_mac_fsm_crc_if`, connecting `can_mac_fsm_tx/can_mac_fsm_rx` and `can_mac_crc` (see Figure 4). Separate data feeds for CC and FD are required because CC and FD frames compute CRC over different bit streams: CC excludes stuff bits while FD includes them [1, Sec. 10.4.2.6]. This dual-feed design also allows the RX path to run both CRC engines in parallel until the frame type is known. The CRC engine is reset by a dedicated `crc_rst` signal from the FSM.

| field                        | type                                | direction                  | description                                                                            |
|------------------------------|-------------------------------------|----------------------------|----------------------------------------------------------------------------------------|
| <code>crc_poly_select</code> | <code>std_logic_vector(1:0)</code>  | <code>fsm -&gt; crc</code> | selects the active CRC polynomial: CRC-15, CRC-17, or CRC-21                           |
| <code>valid_cc</code>        | <code>std_logic</code>              | <code>fsm -&gt; crc</code> | pulsed when <code>data_cc</code> should be accumulated into the CRC-15 engine          |
| <code>valid_fd</code>        | <code>std_logic</code>              | <code>fsm -&gt; crc</code> | pulsed when <code>data_fd</code> should be accumulated into the CRC-17/CRC-21 engines  |
| <code>data_cc</code>         | <code>std_logic</code>              | <code>fsm -&gt; crc</code> | bit value fed to the CRC-15 engine (CC frames: data bits only, no stuff bits)          |
| <code>data_fd</code>         | <code>std_logic</code>              | <code>fsm -&gt; crc</code> | bit value fed to the CRC-17/CRC-21 engines (FD frames: dynamic stuff bits + data bits) |
| <code>crc</code>             | <code>std_logic_vector(20:0)</code> | <code>crc -&gt; fsm</code> | current CRC register value, left-aligned and zero-padded to 21 bits                    |

**1.6.1.2 `can_mac_fsm_tx`** `can_mac_fsm_tx` (Figure 5) orchestrates frame transmission, coordinating the serializer (Section 1.6.1.3), bit stuffer (Section 1.6.1.4), and CRC engine (Section 1.6.1.5). The FSM uses per-field state granularity: rather than a single `s_transmitting_frame` state with function-dispatched field logic, each protocol field (SOF, ID, DLC, data, CRC, ACK, EOF, etc.) has its own dedicated FSM state. This design makes the field boundaries explicit in the state encoding and eliminates the need for frame parameter precomputation functions. The FSM derives `data_len` and `crc_length` from `t_llc_metadata` on frame entry and tracks `bit_count` within each field state.

On each sample-point strobe from `can_pcs_tx` (Section 1.8.1), each frame-field state executes the following sequence:

1. **Monitor the bus.** The sampled bus polarity is compared against the previously transmitted bit to detect errors, ACK, and arbitration loss. In the CAN-FD data phase, the FSM maintains a 32-bit polarity

history shift register for Transmitter Delay Compensation (TDC). Any pending secondary sample point (SSP) observation from the previous bit period is evaluated against this history before the current bit is checked [1, Sec. 7.3.4]. A detected error triggers a transition to `s_error_overload` with the appropriate flag type based on the FCE fault confinement status [1, Secs. 8.1.3–8.1.4]. Arbitration loss during the ID or control fields causes the FSM to stop driving and return to `s_intermission`.

2. **Determine the next bit.** If the bit stuffer has a pending stuff bit (`bs_i.valid`), that takes priority. Otherwise, the polarity is determined by the current state: form bits (SOF, IDE, FDF, reserved, delimiters, EOF) have fixed polarities, while ID, DLC, data, SBC, and CRC bits are sourced from the serializer, metadata, bit stuffer count, or CRC register respectively.
3. **Feed the CRC engine and bit stuffer.** The output polarity is forwarded to the CRC engine (for bits within the CRC-protected region) via separate `data_cc` and `data_fd` feeds, and to the bit stuffer (for bits within the stuffing region). The FSM asserts `fixed_bit_stuffing_en` when entering the SBC/CRC field region of FD frames to switch the bit stuffer from dynamic to fixed mode.
4. **Present the bit at the PCS interface.** The resolved polarity is driven onto the `t_can_mac_pcs_if_m2s` interface. The `use_data_rate` signal is asserted during the CAN-FD data phase to switch the PCS to faster bit timing, and `start_tdc` is pulsed at the FDF bit to initiate transmitter delay compensation [1, Sec. 7.3.4].

**1.6.1.3 can\_mac\_ser\_tx** `can_mac_ser_tx` converts the LLC byte stream into a serial polarity bit stream for the MAC FSM. Its four-state FSM (Figure 6) manages the two-byte configuration handshake, byte fetching, and bit-by-bit serialization. The serializer extracts LLC metadata (IDE, FDF, DLC, FTYP, BRS, ESI) from the two config bytes and registers it in `t_llc_metadata`, which remains stable for the entire frame. The serializer also handles ID field padding - skipping unused bits in the 32-bit ID field for 11-bit base identifiers - and forwards `transfer_status` from the FSM back to the LLC to terminate serialization on error or completion.

**1.6.1.4 can\_mac\_bs** `can_mac_bs` implements both dynamic and fixed bit stuffing for CAN Classic and CAN-FD frames [1, Sec. 10.6]. The same entity is instantiated in both `can_mac_tx` and `can_mac_rx` - on the TX side it inserts stuff bits, while on the RX side the FSM uses its output to detect and remove stuff bits from the received stream.

In **dynamic mode** (`fixed_bit_stuffing_en` = '0'), the stuffer monitors the polarity stream and inserts an inverse-polarity stuff bit after every five consecutive identical bits (ISO 6.6.13.2). In **fixed mode** (`fixed_bit_stuffing_en` = '1'), used for the FD CRC region, one fixed stuff bit (FSB) is inserted on the rising edge of `fixed_bit_stuffing_en`, then one every four real bits (ISO 6.6.13.3.1). Any dynamic stuff bit pending at the mode boundary is suppressed. The stuffer maintains a Gray-coded stuff bit count with parity for the SBC field via `f_to_gray()` and `f_calc_parity()`. The data path diagram is shown in Figure 7.

**1.6.1.5 can\_mac\_crc** `can_mac_crc` provides CRC generation and checking for both CAN Classic and CAN-FD frames. CAN Classic frames use CRC-15, while CAN-FD frames use CRC-17 (data payloads up to 16 bytes) or CRC-21 (data payloads above 16 bytes) [1, Sec. 10.4.2.6]. The same entity is instantiated in both `can_mac_tx` (for generation) and `can_mac_rx` (for checking). The FSM selects the appropriate polynomial via the `crc_poly_select` field (Table 13). The data path diagram is shown in Figure 8.

Three parallel `gen_crc` instances run continuously on separate data feeds: `data_cc` drives CRC-15 via

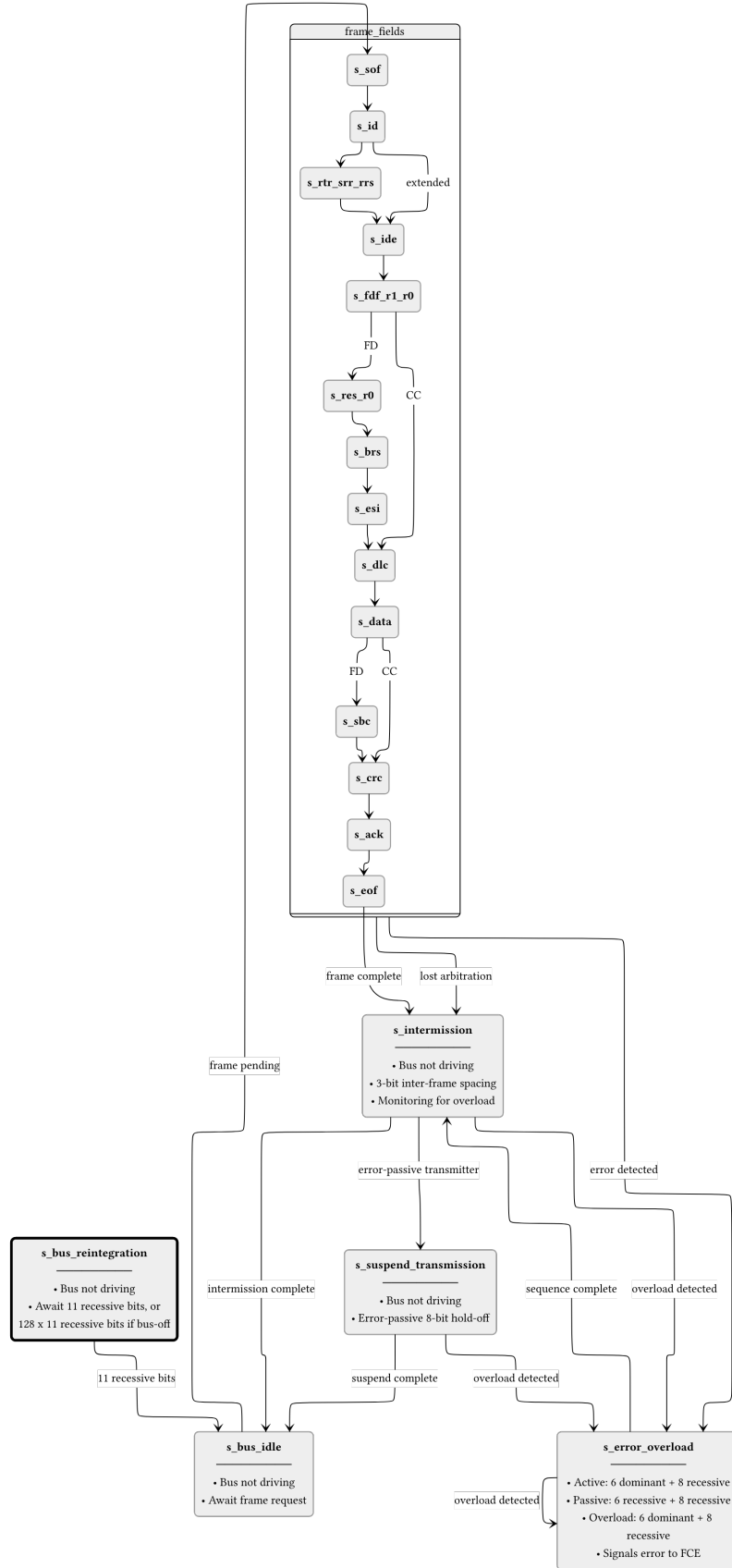


Figure 5: can\_mac\_fsm\_tx FSM with per-field state granularity. Frame-field states (s\_sof through s\_eof) each handle one protocol field, with bit\_count tracking position within the field. After a successful frame or arbitration loss the FSM passes through s\_intermission before returning to s\_bus\_idle. Detected errors branch to s\_error\_overload, which drives either an active (dominant) or passive (recessive) error flag depending on FCE status. The dashed box groups the frame-field states for clarity.

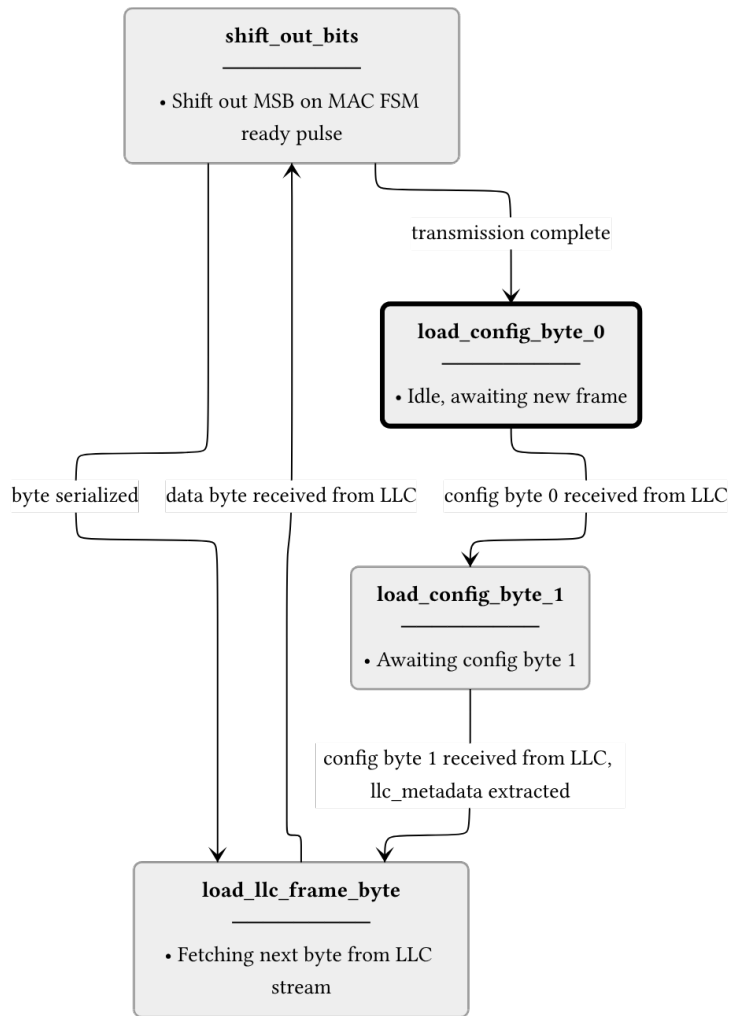


Figure 6: can\_mac\_ser\_tx FSM. The serializer accepts LLC frame bytes over a ready/valid handshake and shifts them out MSB-first to the MAC FSM one bit per ready pulse. LLC metadata is extracted from the two config bytes and registered in t\_llc\_metadata for use by the FSM throughout the frame.

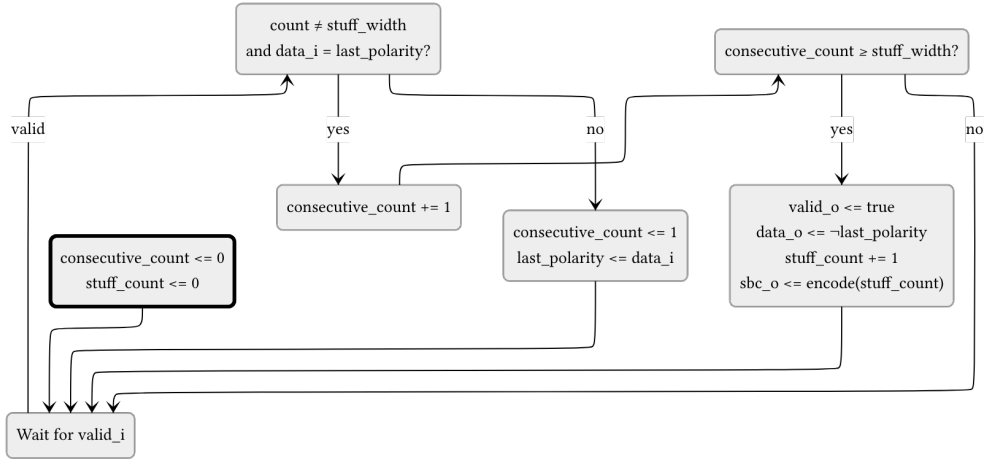


Figure 7: can\_mac\_bs data path diagram. When valid and data = last\_polarity, consecutive\_count increments. Otherwise it restarts at 1 with last\_polarity updated to data. When consecutive\_count reaches stuff\_width (5 in dynamic mode, 4 in fixed mode), valid and inverted data are driven and stuff\_count increments. The to\_gray() + calc\_parity() block encodes stuff\_count into the stuff\_bit\_count output. The external rst signal resets consecutive\_count and stuff\_count to zero. The fixed\_bit\_stuffing\_en signal selects between dynamic and fixed modes. All outputs are registered.

valid\_cc, while data\_fd drives both CRC-17 and CRC-21 via valid\_fd. This dual-feed architecture is necessary because CC and FD frames compute CRC over different bit streams (CC excludes stuff bits; FD includes them), and the RX path does not know which CRC engine to use until after the frame type has been determined. The output multiplexer selects the active engine's result based on crc\_poly\_select and left-aligns it to the common 21-bit output width.

### 1.6.2 can\_mac\_rx

can\_mac\_rx receives the bit stream from the PCS, performs bit destuffing and CRC verification, and reconstructs the LLC frame for delivery to can\_llc\_rx [1, Sec. 6.6]. Unlike the TX path, the RX path has no separate deserializer module. The RX FSM (can\_mac\_fsm\_rx) stores received bits directly into an internal llc\_frame array during reception and streams the completed frame byte-by-byte to the LLC during the quiet phase (after EOF and intermission) using a dedicated Avalon-ST streaming process.

The can\_mac\_rx wrapper instantiates three components:

1. **can\_mac\_fsm\_rx**: Frame reception FSM with 17 per-field states mirroring the TX FSM structure
2. **can\_mac\_bs**: Shared bit stuffer entity, reused for destuffing
3. **can\_mac\_crc**: Shared CRC engine entity, reused for CRC checking

The RX FSM (Figure 9) validates SBC and CRC fields during reception, detects form errors (reserved bits, CRC mismatch, delimiter errors), and drives ACK dominant during the ACK slot (1 bit for CC, 2 bits for FD). It reports errors to the FCE through the same t\_can\_mac\_fce\_if\_m2s interface used by the TX FSM.

### 1.6.3 can\_mac Unified Wrapper

can\_mac is a structural wrapper that instantiates can\_mac\_tx, can\_mac\_rx, and can\_fce (Section 1.7), wiring the FCE internally so that the wrapper exposes only LLC and PCS interfaces for each path plus the FCE's LLC and PCS interfaces. TX and RX have separate PCS interfaces (half-duplex; only one path drives the bus at a time). The TX and RX FCE output records (t\_can\_mac\_fce\_if\_m2s) are merged by OR-

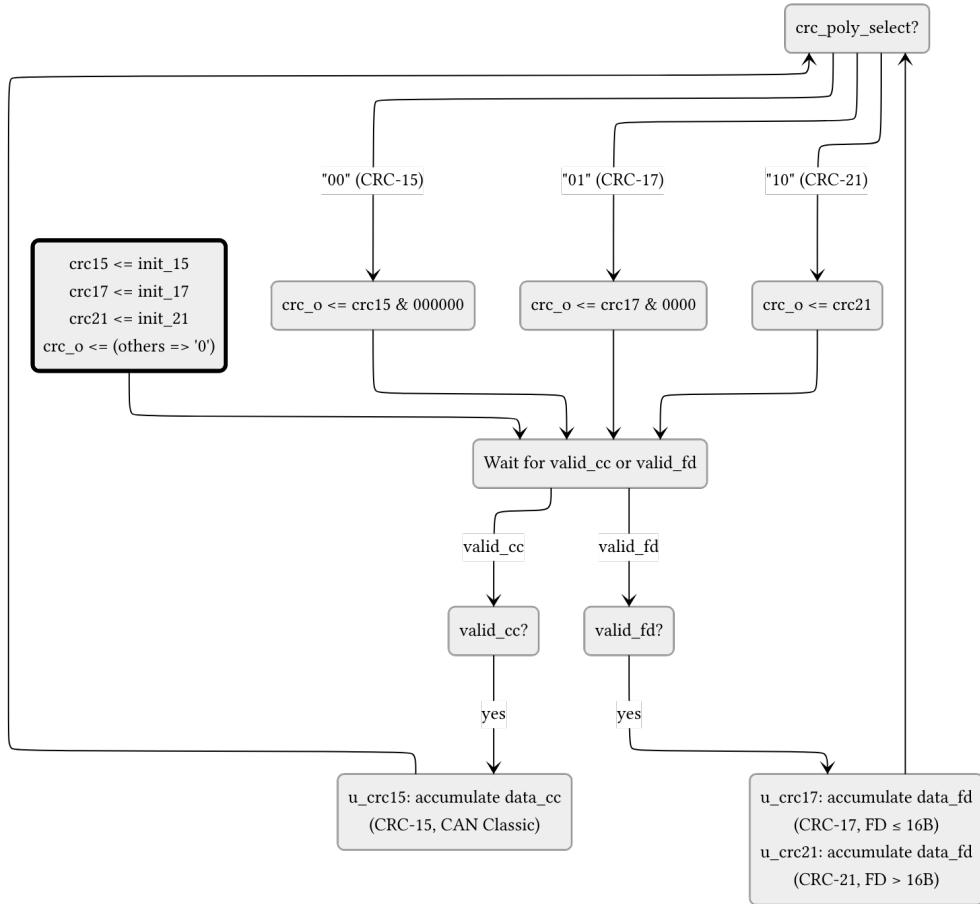


Figure 8: `can_mac_crc` data path diagram. Three parallel `gen_crc` instances accumulate continuously: `data_cc` drives CRC-15 via `valid_cc`, while `data_fd` drives both CRC-17 and CRC-21 via `valid_fd`. The output register selects the active engine result based on `crc_poly_select` and left-aligns it to 21 bits. The external `rst` signal reinitializes all three engines and the output register.

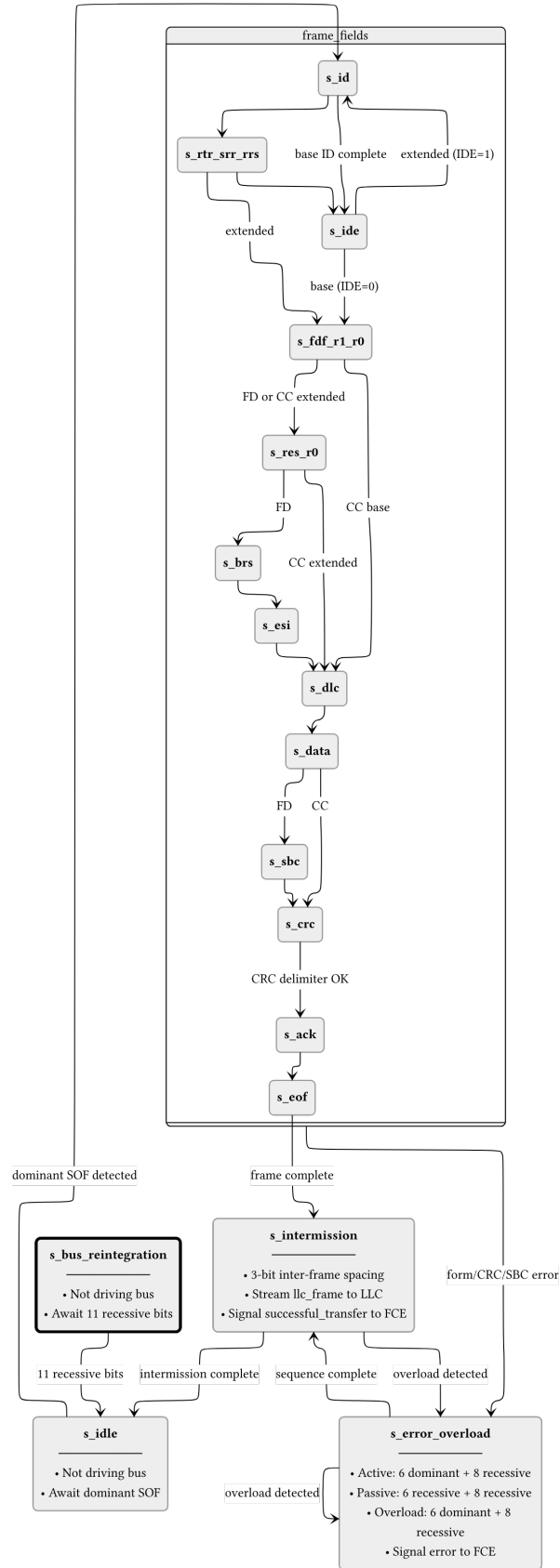


Figure 9: can\_mac\_fsm\_rx FSM with per-field state granularity. The RX FSM mirrors the TX FSM structure (Figure 5) but replaces transmission logic with reception, storage, and validation. On detecting a dominant SOF in s\_idle, the FSM enters s\_id and stores received bits into an internal llc\_frame array. After EOF, the FSM transitions through s\_intermission and a dedicated streaming process transfers the stored frame to the LLC. Form errors (reserved bit violations, CRC mismatch, delimiter errors) and SBC mismatches trigger s\_error\_overload. The dashed box groups the frame-field states for clarity.

ing all fields before feeding the FCE input - the half-duplex guarantee ensures no simultaneous assertions from both paths. The FCE's response (`t_can_mac_fce_if_s2m`, carrying `error_passive_request` and `error_active_request`) is fanned back to both TX and RX paths.

## 1.7 FCE Sub-layer

The Fault Confinement Entity (`can_fce`) implements the error state machine and counter management specified in [1, Secs. 8.1.3–8.1.4]. It maintains two error counters - TEC (Transmitter Error Counter, 0-511) and REC (Receiver Error Counter, 0-255) - and transitions between three states: `s_error_active` (normal operation), `s_error_passive` (TEC or REC > 127), and `s_bus_off` (TEC > 255).

Counter updates follow the ISO 8.1.4.2 rules: TEC increments by 8 on TX errors (unless `counters_unchanged` is asserted), decrements by 1 on successful TX. REC increments by 1 on RX errors during non-error-flag phases, by 8 on primary errors or error-flag-phase errors, and decrements by 1 or clamps to 127 on successful RX. The FCE state machine (Figure 10) transitions between error-active, error-passive, and bus-off based on counter thresholds. Bus-off recovery requires counting 128 idle conditions (11 consecutive recessive bits each) from the PCS via the `idle_condition` signal, or a `normal_mode_request` from the LLC.

## 1.8 PCS Sub-layer

Handles bit timing and synchronization. It generates the sample point (SP) and secondary sample point (SSP) strobes. It provides bit-level monitoring data to the FCE to detect synchronization and timing errors.

### 1.8.1 `can_pcs_tx`

`can_pcs_tx` handles bit timing and Transmitter Delay Compensation (TDC) for the TX path [1, Secs. 7.2–7.3]. It generates the sample point (SP) strobe used by the MAC FSM to advance frame state, and - when TDC is configured - a secondary sample point (SSP) strobe for data-phase bit monitoring. The FSM (Figure 11) has four states reflecting the two bit rates and the TDC measurement window. The `measuring_delay` state determines the Transmitter Delay Compensation Value (TDCV, [1, Sec. 7.3.4]) by observing the TX-to-RX propagation delay, which together with the configured TDCO offset defines the SSP position for subsequent data-phase bits.

### 1.8.2 `can_pcs_rx`

`can_pcs_rx` implements bit timing and synchronization for the RX path. It performs hard and soft synchronization on the incoming bus signal and generates the sample point strobe used by the MAC RX layer to latch each received bit [1, Secs. 7.2–7.3].

---

[1] International Organization for Standardization, *Road vehicles — controller area network (CAN) — Part 1: Data link layer and physical coding sublayer*. 2024.

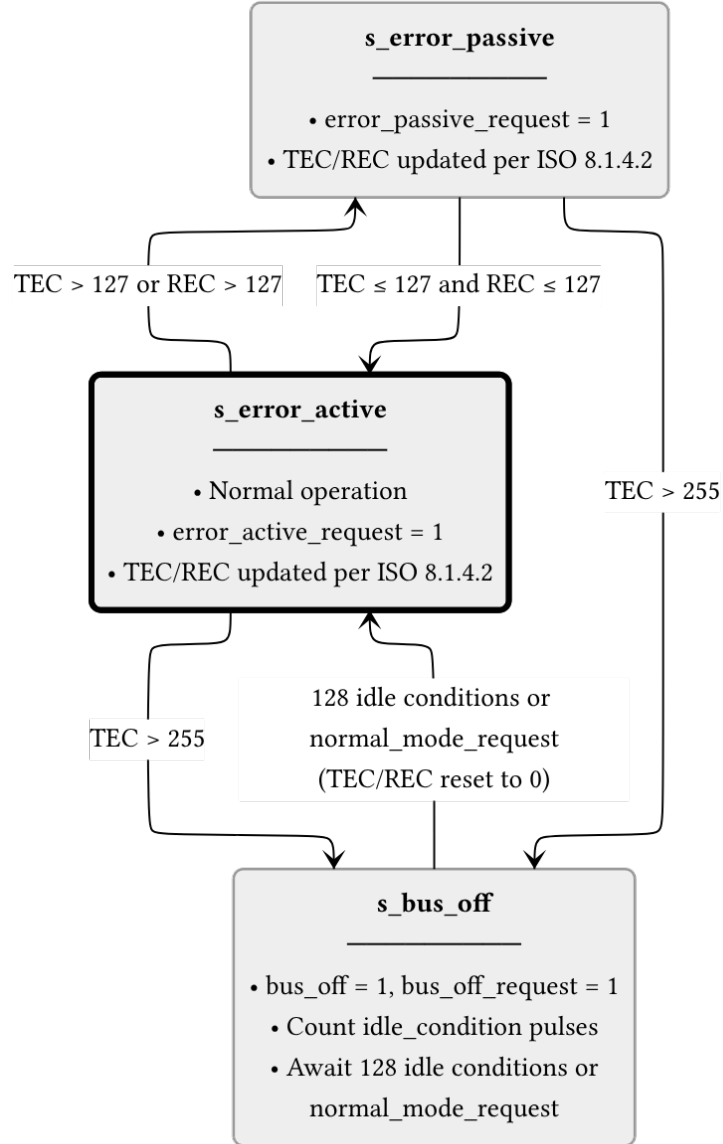


Figure 10: can\_fce FSM (ISO 8.1.4.1, Figure 43). The s\_error\_active state is the normal operating mode. When either TEC or REC exceeds 127, the FCE transitions to s\_error\_passive and asserts error\_passive\_request to both MAC paths. If TEC exceeds 255, the node enters s\_bus\_off: the FCE issues bus\_off\_request to the PCS and bus\_off to the LLC. Recovery from bus-off requires 128 idle conditions (each 11 consecutive recessive bits) counted from the PCS, or a normal\_mode\_request from the LLC, after which the counters are reset and the FCE returns to s\_error\_active.

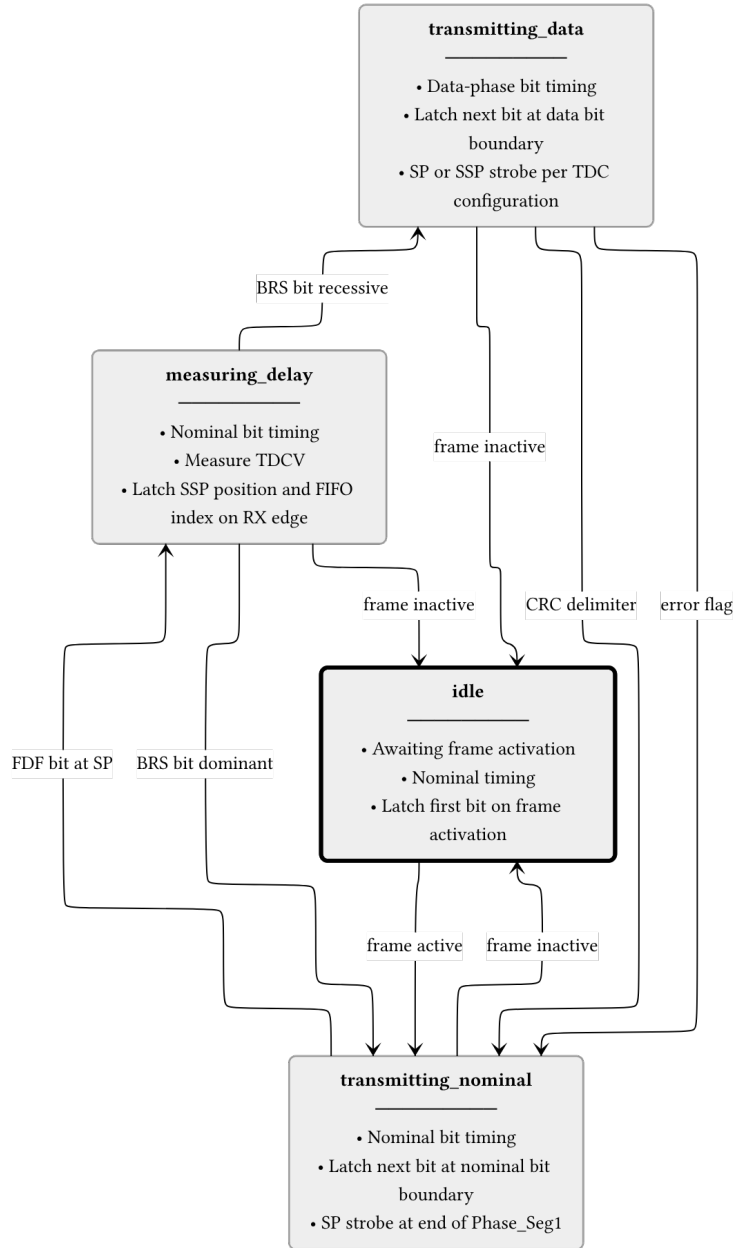


Figure 11: can\_pcs\_tx FSM. The measuring\_delay state is entered on the FDF sample point to measure TDCV (Transmitter Delay Compensation Value, [\[1\]](#), sec. 7.3.4)]. The BRS bit boundary determines whether data-phase timing is used. All non-idle states return to idle when the frame becomes inactive.